Ontology as a Hyperparameter: Task-Aware Knowledge Graph Construction Using LLMs

Jan Frąckowiak[1] Piotr Wójcik, PhD[1]
Michał Sierakowski, PhD[2]

[1]Faculty of Economic Sciences, University of Warsaw

[2]Faculty of Mathematics, Informatics, and Mechanics, University of Warsaw

Abstract

This project investigates the construction of task-aware knowledge graphs (KGs) from text, treating graph structure as an optimization target rather than a fixed representation. Existing frameworks such as Neo4j-based pipelines and LLM-driven tools (e.g., LlamaIndex) focus on constructing large, general-purpose graphs without incorporating feedback from downstream tasks, despite evidence from evaluation frameworks (e.g., KGrEaT, GEval, KGBench) that KG utility is task-dependent. The research addresses this gap by proposing a feedback-driven approach in which ontology is treated as a tunable component influencing predictive performance. The study uses document-based datasets linking text to measurable outcomes, including financial news (FNSPID) paired with stock price movements. LLM agents are employed for ontology evolution and triple extraction, enabling scalable generation of multiple KG variants. Graph-derived features are then computed over temporal windows and used in predictive models, whose performance is fed back to the agents to refine ontology design, closing the loop between graph construction and downstream task. Preliminary results indicate that graph structure strongly influences predictive outcomes, with adaptive ontology updates leading to significant improvements.

Contents

1 Introduction

Representing knowledge in structured forms that intelligent systems can query and reason over is a foundational challenge of artificial intelligence. Knowledge graphs (KGs) have become the attractive medium for this purpose, encoding entities, relationships, and semantic descriptions extracted from text to support applications from question answering and recommendation to predictive modelling (Ji et al., 2021). The ability to construct KGs from unstructured text at scale has been substantially advanced by large language models (LLMs), which enable flexible extraction of entities and relations without handcrafted rules (Mo et al., 2025). Despite this progress, a fundamental tension persists in KG construction: ontology and graph structure are typically specified *a priori* and evaluated independently of the tasks for which the graph is ultimately used.

Existing KG construction tools — from manual ontology engineering to LLM-driven frameworks such as LlamaIndex and LangChain — treat the ontology as a fixed design choice established before construction begins and unchanged throughout. Evaluation has historically relied on structural quality metrics independent of any downstream task (Paulheim, 2017), and while frameworks such as KGrEaT (Heist et al., 2023) and KGBench (Bloem et al., 2021) demonstrate that task-based evaluation is feasible, we could not find work demonstratating use of downstream performance signal to steer KG ontology evolution during the construction process itself.

This paper aims to address the gap between graph construction and downstream task performance by introducing a feedback-driven framework in which the KG ontology is treated as a tunable hyperparameter. Rather than fixing the schema in advance, we propose a closed-loop system:

(i) An Ontology Agent proposes candidate ontology configurations.

(ii) A KG Builder Agent constructs knowledge graphs from text under each configuration.

(iii) Graph-derived features are computed and evaluated on a downstream task.

(iv) The resulting task performance score is returned as a feedback signal to choose the best candidate graphs and guide the next ontology revision.

This loop repeats until performance converges.

We instantiate this framework on the task of next-day stock price movement prediction using the FNSPID financial news dataset Dong et al. (2024), a corpus of financial news articles linked to NASDAQ price time series. This domain offers a clear and measurable downstream objective, dense entity interactions, and a natural temporal structure that motivates subgraph-based feature extraction. The framework is designed to be domain-agnostic and is extensible to other settings.

Contributions. This paper makes the following contributions:

1. We propose a feedback-driven KG construction framework that treats ontology as a hyperparameter optimized for downstream task performance, closing the loop between graph construction and task evaluation.

2. We introduce a structured feature representation for temporal KG subgraphs — comprising node type histograms, relation type histograms, metapath counts, temporal novelty statistics, and topology features — that supports lightweight downstream task evaluation without requiring graph neural networks.

3. We provide empirical evidence on the FNSPID dataset that task-aware ontology adaptation improves downstream task performance, enables more compact graph representations without sacrificing performance, and produces more informative graph structures over successive iterations.

Paper organisation. Section 2 reviews related work on KG construction, task-based KG evaluation, and task-aware graph learning. Section 3 states the research hypotheses formally. Section 4 describes the proposed framework in detail. Section 5 presents the experimental setup, and Section 6 reports results. Section 7 discusses findings and limitations, and Section 8 recapitulates findings.

2 Related Work

The advent of large language models has substantially lowered the barrier to KG construction at scale, and the popularity of this approach has led to a proliferation of tools and frameworks for LLM-augmented KG construction. Pan et al. (2024) survey this landscape, classifying LLM-augmented construction methods by the role of the LLM — entity discovery, relation extraction, end-to-end generation, or knowledge distillation. Among open-source frameworks, LlamaIndex, LangChain, and Neo4j are the dominant providers. LlamaIndex (Liu, 2022) offers three extraction strategies for property graph construction (LlamaIndex, 2024): a schema-free extractor in which the LLM assigns entity and relation types freely (`SimpleLLMPathExtractor`); a strict schema extractor that enforces a user-defined list of allowed types (`SchemaLLMPathExtractor`); and a guided extractor that accepts an initial type list as non-binding hints while permitting the LLM to introduce novel types (`DynamicLLMPathExtractor`). LangChain's `LLMGraphTransformer` (Chase, 2022; Bratanic, 2024) follows a similar pattern. In both modes users may supply `allowed_nodes` and `allowed_relationships` as schema constraints, with a strict flag that discards extractions violating the schema. The Neo4j LLM Knowledge Graph Builder (Neo4j Labs, 2024; Neo4j, 2024) offers three schema modes: a user-supplied schema of node types, relationship types, and valid source–relation–target triples; an auto-extracted schema inferred

from the input text in a single upfront LLM call and held fixed thereafter; and a free mode with no schema.

These tools have seen wide adoption but share a fundamental design choice: the ontology — node types, relation types, and constraints — is specified arbitrarily and remains fixed for the duration of construction or even sometimes is built ad hoc by the LLM without an explicit schema.

Several academic approaches try to bridge triple extraction quality and task specification. Mo et al. (2025) propose KGGen, which extracts KGs from plain text using language models with improved entity and relation normalisation: an iterative clustering step collapses the near-duplicate relation types that open extraction produces, yielding denser and more connected graphs. Mynarz and Haniková (2023) propose a test-driven approach in which competency questions, formalised as SPARQL queries, drive extraction — providing a form of task specification without a quantitative feedback loop.

Li et al. (2025) provide the most comprehensive survey to date of LLMs across the ontology engineering lifecycle, reviewing 30 publications and 41 experiments spanning requirements specification, conceptualisation, encoding, ontology matching, and maintenance. The distribution of effort is stark: ontology implementation accounts for 63.4% of studied experiments, while ontology revision and maintenance has been addressed by a single study (2.4%). The survey names the consequences directly: "support across the ontology lifecycle is uneven, with maintenance particularly underexplored," and finds that "evaluation practices are fragmented, as existing quantitative metrics fail to fully capture performance" (Li et al., 2025); across all reviewed studies, precision, recall, and F1 are applied against ontology benchmarks such as the Ontology Alignment Evaluation Initiative (OAEI), not against a downstream application.

The survey closes by identifying ontology revision and maintenance as an open direction: "Future research directions may include developing hybrid learning pipelines that allow LLMs to continuously integrate domain updates ... thereby supporting ongoing validation and refinement based on domain-specific feedback" (Li et al., 2025).

The evaluation of knowledge graphs has historically been conducted through intrinsic, structural quality metrics. Paulheim (2017) survey methods for KG completion and error detection, evaluating them against structural criteria such as precision and recall on missing type and relation assertions. More recently, Paulheim (2024) draw an explicit distinction between this intrinsic mode of evaluation and task-based evaluation — assessing a KG by its utility on a concrete downstream application. This distinction is central to the motivation of the present work: a graph may score well on structural criteria while failing to support a specific predictive task.

Several frameworks try to standardise task-based KG evaluation. KGBench Bloem et al. (2021) provides a suite of relational and multimodal machine learning tasks benchmarked on curated KG datasets. KGrEaT Heist et al. (2023) enables systematic com-

parison of KG variants across classification, regression, and recommendation tasks; the framework explicitly supports comparison of "different versions of a KG during its life cycle," precisely the kind of comparison our feedback loop performs automatically. LLM-KG-Bench **?** focuses specifically on the quality of LLM-generated graphs. Together, these frameworks confirm that task utility cannot be inferred from structural properties alone — yet none of them feeds performance signals back into the construction process itself.

The closest prior work to our approach concerns task-aware graph construction. Tsang et al. (2025) introduce AutoGraph-R1, the first framework to directly optimise KG construction for task performance using reinforcement learning: an LLM constructor is trained as a policy whose reward derives from the graph's functional utility in a downstream RAG retrieval pipeline. AutoGraph-R1 shares with our approach the core principle of aligning construction with a downstream objective; the critical differences are that its reward derives from retrieval performance in a RAG pipeline rather than a node-level classifier, the RL policy optimises extraction density and triple relevance within an open-schema setting rather than selecting among ontology configurations, and there is no mechanism for explicit schema evolution.

Cucumides and Geerts (2025) propose constructing graphs from tabular and relational data to maximise GNN performance on a target task, tailoring graph topology through iterative process where each attribute's predictive relevance is assessed. However, their setting involves structured tabular input rather than unstructured text, and the construction is not driven by an evolving ontology.

The financial domain chosen for this study provides a natural testbed for task-conditioned KG construction. Financial news significantly influences market dynamics (Dong et al., 2024), and a long line of work has explored how to represent its informational content as structured graphs for downstream prediction. Ding et al. (2015) demonstrate that event-driven representations of financial news — structured as subject–predicate–object triples capturing who did what to whom — yield strong signals for short- and medium-term stock price movement when combined with deep neural networks. Subsequent work confirms and extends this finding: Ding et al. (2016) show that enriching event representations with knowledge graph embeddings further improves prediction accuracy, and attention-based models over news sequences (Hu et al., 2018) and financial social media text (Xu and Cohen, 2018) demonstrate the breadth of the predictive signal across sources and markets.

Arun et al. (2025) introduce FinReflectKG, a pipeline for constructing financial KGs from SEC 10-K filings using a pre-configured schema of entity and relationship types specific to financial regulatory documents. An LLM-based reflection component iterates to correct extraction errors within this fixed schema until no issues are detected or a step budget is exhausted.

Taken together, the literature reveals a consistent gap. Evaluation frameworks con-

firm that KG utility is task-dependent and cannot be inferred from structural properties alone (Heist et al., 2023; Bloem et al., 2021). Task-aware construction methods demonstrate that graph structure can be aligned with downstream objectives (Cucumides and Geerts, 2025; Tsang et al., 2025), and many researchers assume that event-driven representations of financial news carry strong predictive signals for market classification (Ding et al., 2015, 2016; Hu et al., 2018; Xu and Cohen, 2018). Yet no existing work closes the loop: temporal KG methods evolve facts within a fixed schema (**?**); task-aware methods adapt topology or extraction reward without revising the ontology (Cucumides and Geerts, 2025; Tsang et al., 2025); financial KG systems such as FinReflectKG (Arun et al., 2025) iterate within a pre-configured schema; and ontology engineering surveys identify schema maintenance as the most underexplored stage of the lifecycle (Li et al., 2025). No existing method treats the KG ontology — the choice of node types, relation types, and constraints — as a hyperparameter to be optimised through a closed feedback loop between a downstream classifier and the construction process. The present paper addresses this gap directly.

3 Research Hypotheses

The proposed framework rests on the premise that knowledge graph ontology is not a neutral design choice but an active determinant of downstream predictive performance. If this is true, then iteratively adapting the ontology in response to task feedback should yield measurable improvements — both in accuracy and in the structural properties of the resulting graph. We formalise this premise through three testable hypotheses, each targeting a distinct dimension of the expected benefit: predictive accuracy, graph efficiency, and structural informativeness.

Hypothesis 1. *Task-aware knowledge graph construction improves prediction accuracy in node-level price movement classification compared to both (i) a static, unoptimized knowledge graph built once under a fixed base ontology and (ii) a graph-free baseline using mean-pooled article text embeddings.*

Ontology configurations that are better aligned with the prediction objective should produce graph-derived features with higher discriminative power than both an unoptimized graph and raw article text alone. H1 is evaluated by comparing AUC and F1-score of the downstream classifier trained on features extracted from task-aware KG variants against the static-ontology baseline, a graph built once under the fixed base schema of Section 4.2 and never revised, and against the text-only baseline, which uses mean-pooled article embeddings with no graph at all (Section 5.2, Section 6).

Hypothesis 2. *Task-aware optimization enables smaller knowledge graphs while maintaining or improving downstream task performance.*

By discarding ontology elements that do not contribute to predictive performance, the feedback loop should converge on more compact representations. H2 is evaluated by tracking the number of nodes and edges per iteration alongside classifier AUC, testing whether the task-aware variant achieves a favourable size–performance tradeoff relative to the unconstrained baseline (Section 6).

Hypothesis 3. *Iterative, task-driven knowledge graph construction produces more informative graph structures and facilitates the identification of effective ontology configurations.*

Successive ontology revisions guided by performance feedback should increase the diversity and predictive relevance of extracted relation types. H3 is evaluated by examining the ontology evolution trace — which relation types are added, retained, or removed across iterations — and by measuring feature entropy and metapath diversity over successive construction steps (Section 6).

4 Methodology

4.1 Framework Overview

We treat the KG ontology as a hyperparameter optimised for downstream task performance through a closed feedback loop. An ontology $\mathcal{O}$ specifies the permitted node types, relationship types, and valid source–relation–target patterns used to extract a typed knowledge graph $\mathcal{G}(\mathcal{O})$ from a text corpus. Given a feature extractor f and a downstream classifier $\mathcal{M}$, the loop iterates:

$$\mathcal{O}^{(t+1)} = \text{EvolveAgent}\Big(\mathcal{O}^{(t)},\ \text{AUC}\Big(\mathcal{M}\Big(f\Big(\mathcal{G}\Big(\mathcal{O}^{(t)}\Big)\Big)\Big)\Big)\Big) \tag{1}$$

At each step t, the Evolution Agent proposes K candidate ontologies derived from the current best. For each candidate, a graph is constructed from the input corpus, structural features are computed per trading day, and an XGBoost classifier is trained and evaluated on a downstream prediction task. The candidate with the highest validation AUC is selected as the seed for the next step; the loop terminates when the step budget is exhausted. Figure 1 illustrates the complete loop.

4.2 Ontology Representation and Base Schema

An ontology candidate $\mathcal{O}$ is represented as $(\mathcal{T}_N, \mathcal{T}_R, \mathcal{C})$, where $\mathcal{T}_N$ is a set of typed node definitions (label plus property schema), $\mathcal{T}_R$ is a set of relationship type strings, and $\mathcal{C}$ is a set of valid (source, relation, target) triple patterns.

The base ontology $\mathcal{O}^{(0)}$ is fixed and minimal: $\mathcal{T}_N = \{$`Company`, `Article`, `Author`, `Day`$\}$; $\mathcal{T}_R = \{$`MENTIONS`, `RELATES_TO`, `WRITTEN_BY`, `PUBLISHED_ON`$\}$; and three base patterns:

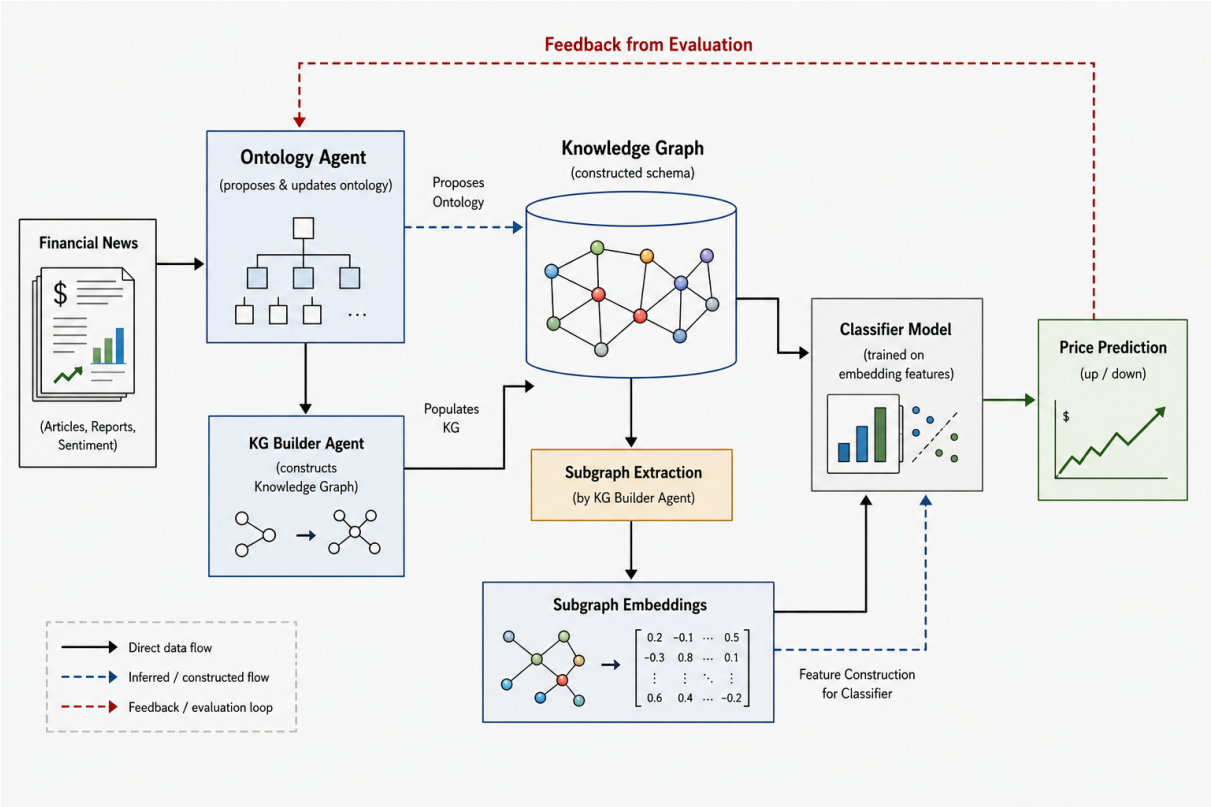

Figure 1: Overview of the feedback-driven KG construction framework. The Ontology Agent proposes candidate schemas; the KG Builder Agent constructs the graph from financial news under each schema; subgraph features are extracted and fed to a downstream classifier; the resulting AUC is returned as a feedback signal to guide the next round of ontology proposals.

- Article→MENTIONS→Company
- Article→WRITTEN_BY→Author
- Article→PUBLISHED_ON→Day

This base structure is built once without LLM extraction and shared by all subsequent candidates.

Dataset and storage. The framework is instantiated on the FNSPID dataset (Dong et al., 2024), a corpus of financial news articles linked to NASDAQ stock price time series. Each trading day receives a binary label $y_d = \mathbf{1}[\text{next-day return} > 0]$ stored as a property on the corresponding `Day` node. All graph data are stored in a Neo4j AuraDB instance.

4.3 KG Construction and Candidate Isolation

LLM extraction. For each article, an `IncrementalArticleKGMutator` invokes an LLM with a structured prompt specifying the candidate ontology and requesting a JSON response of the form `{"nodes": [...], "relationships": [...]}`. Extracted nodes carry

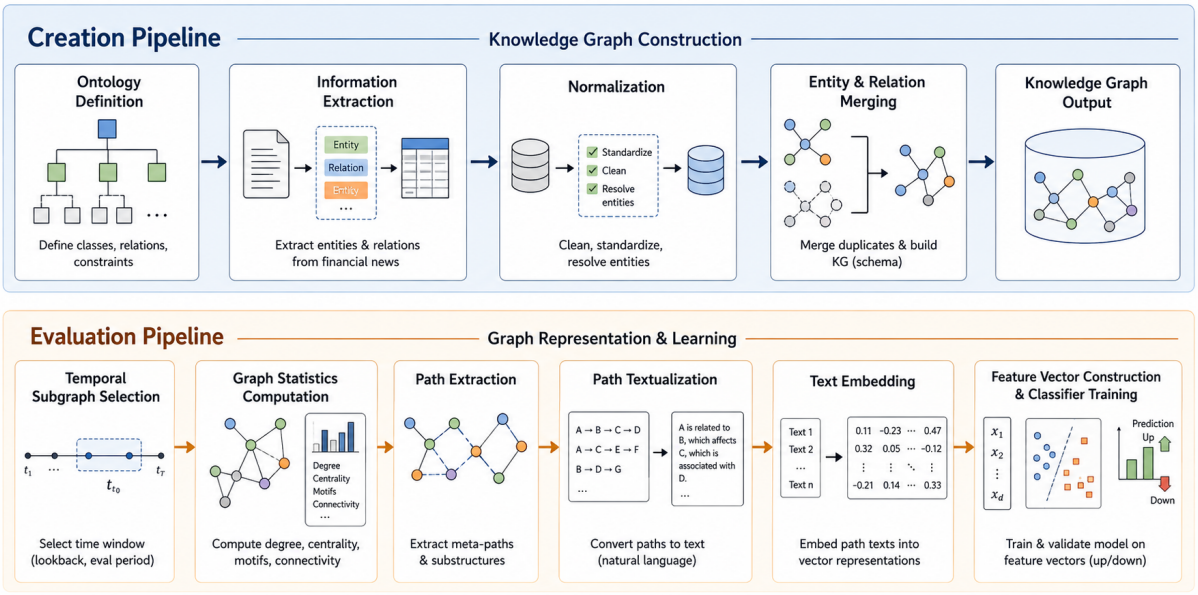

Figure 2: Pipeline detail. *Creation Pipeline* (top): ontology definition, LLM-based information extraction, entity normalisation, and merging into the shared Neo4j graph. *Evaluation Pipeline* (bottom): temporal subgraph selection, graph statistics and metapath computation, path extraction and textualization, sentence embedding, and downstream classifier training.

a label, key, and a property dictionary; relationships carry a type, source key, and target key. Company nodes are keyed by stock ticker (`MSFT`, `TSLA`); all other types are keyed by a lowercase name slug to promote cross-article deduplication.

Candidate isolation. Multiple ontology candidates are evaluated within a single shared Neo4j database to avoid redundant storage. Isolation is enforced through a `candidate_tags` list property on every node and relationship. When building candidate c, newly extracted entities may link only to entities whose `candidate_tags` intersect with the accepted set — comprising the base structure and the winners of all previous steps. Entities from sibling candidates at the same step cannot be cross-linked. After evaluation, the tags of losing candidates are pruned from the graph.

Entity deduplication. After each candidate build, a two-pass APOC `mergeNodes` procedure collapses duplicates: first by matching label and lowercased name, then by matching label and uppercased canonical key. Properties and relationships are merged rather than deleted, so no extracted information is lost.

Temporal integrity. To prevent look-ahead data leakage, each article's chain extraction uses the article's own publication date as the upper temporal boundary. A `first_seen` property records the earliest article date at which an entity appeared; all feature queries exclude entities whose `first_seen` is later than the evaluation day.

4.4 Ontology Evolution Agent

The Evolution Agent is an LLM (OpenAI GPT-5-mini) prompted to propose a modified ontology schema given the current best schema and its downstream performance metrics. The agent operates in two phases:

Phase 1 — entity vocabulary expansion. In early steps, the agent selects new entity types from a predefined candidate pool: `Market`, `Sector`, `Index`, `Deal`, `Contract`, `Regulation`, `Product`, `Event`, `Fund`, `Insider`, `Executive`, `Analyst`, and others. For each type added, the agent must propose at least one relationship connecting it to existing types; existing node and relationship types are preserved.

Phase 2 — relationship specialisation. Once the entity vocabulary is sufficiently rich, the agent replaces generic relationship types with alternatives. For example, `RELATES_TO` may be specialised to `COMPETES_WITH`, `PARTNERS_WITH`, `ACQUIRES`, or `INVESTED_IN`. Specific relationship types produce more informative paths and denser subgraph feature signals.

Prompt and feedback signal. The prompt provides the current schema as JSON, the AUC and F1 scores of the previous best candidate, and the maximum relationship-chain depth found during feature extraction — a proxy for graph connectivity. The performance signal is interpreted directionally: AUC at or below 0.5 indicates that current relationships are likely too generic to discriminate price movements. The LLM response is validated against strict JSON schema.

At each step t, K candidates are proposed independently, built and evaluated in sequence, and ranked by validation AUC; the winner becomes the reference in step $t + 1$ and losing candidates have their tags pruned.

4.5 Feature Engineering

Prior work on heterogeneous information networks (HINs) (Shi et al., 2017) shows that typed paths — metapaths (Sun et al., 2011; Dong et al., 2017) — capture relational semantics between node pairs and support downstream classification. Knowledge graph embedding methods (Wang et al., 2017; Ji et al., 2021) and temporal extensions such as Know-Evolve (Trivedi et al., 2017) operate under the assumption that the schema (ontology) is given and fixed: feature engineering, embedding, or message-passing then operates over the resulting typed structure. In our framework the schema is the object being optimised, so the feature representation must be directly comparable across candidates with different ontologies.

Why not learned embeddings. Graph neural network and knowledge graph embedding methods are ill-suited for this evaluation setting for at least three reasons.

Temporal look-ahead. Computing node embeddings on a temporal graph without information leakage requires evaluating the model on graph snapshots taken at each day in the training set, because a GNN model or other embeddings (such as HOPE or FastRP) estimated on the full graph encodes future edges into past node representations. Producing leak-free embeddings for $K \times T$ candidate–step combinations at every evaluation day would be computationally expensive. Moreover, embeddings estimated on day-specific subgraphs would lack stability across temporal slices: the embedding space shifts as the subgraph topology changes, making cross-day feature vectors incomparable even for the same candidate.

Schema instability across candidates. Each ontology candidate introduces a different set of node types and relationship types. A relational GNN (Schlichtkrull et al., 2018) or other embedding model estimated on one candidate's type vocabulary and topology cannot be applied to another: relation-type parameters are semantically tied to the schema under which they were fitted.

Evaluation noise. Independent model fitting per candidate also introduces training variance — from random initialisation and mini-batch ordering — that confounds the AUC comparison between candidates and corrupts the ontology selection signal.

The two feature blocks described below avoid these problems: they are computed locally within each day's lookback window, require no model fitting, are deterministic given the graph state, and produce vectors of fixed dimension regardless of the ontology.

Path embeddings. For each trading day d, relationship chains of length 5–6 hops are extracted from entities mentioned in articles published within a lookback window of k days. Each chain records the alternating sequence of node and relationship types encountered along the path; infrastructure edges (`PUBLISHED_ON`, `WRITTEN_BY`, `MENTIONS`) are excluded so that only domain-relevant relationships contribute. The type sequence of each chain is serialised as a text string and encoded with a sentence encoder (Reimers and Gurevych, 2019) (`all-MiniLM-L6-v2`); the resulting embeddings are mean-aggregated across chains within each article and then across articles in the window, yielding a single 384-dimensional path vector $\mathbf{p}_d \in \mathbb{R}^{384}$. As the ontology evolves and relationship types become more semantically specific, the chain strings become more discriminative, directly linking ontology quality to feature quality.

Subgraph histograms. An alternative, purely count-based block avoids embedding chain text altogether. For entities mentioned in the lookback window, three hash-bucketed histograms are built by deterministically mapping tokens to buckets via truncated MD5 hashing — the first 8 hex digits of the token's MD5 digest are read as an integer and

reduced modulo the bucket count, giving a bucket index in $[0, \text{buckets})$ regardless of the token's length or vocabulary: a node-type histogram (16 buckets), a relationship-type histogram (24 buckets, restricted to relationships tagged with the ontology candidate under evaluation), and a histogram of typed 2- and 3-hop metapaths (48 buckets, node-and-relationship-type sequences such as `Company>MENTIONS>Article>WRITTEN_BY>Author`). Because tokens are hashed rather than looked up in a fixed vocabulary, the vector dimensionality is identical across candidates regardless of how many distinct types each ontology introduces. The bucket a given token maps to is otherwise arbitrary, but the mapping is fixed for a token across days and steps, so the resulting counts are a consistent, reproducible signal for the classifier to exploit even though individual buckets carry no inherent semantic label. Eight further temporal-novelty statistics summarise the window without hashing: article count, mentioned-entity count, new/recurring/undated entity counts (relative to each entity's first mention date, enforcing the same temporal integrity as the topology features below), the resulting novelty and recurrence ratios, and entities per article. Concatenating these four components yields a 96-dimensional subgraph vector $\mathbf{s}_d \in \mathbb{R}^{96}$.

Topology features. For each entity mentioned in the lookback window, eight local graph statistics are computed against the graph state up to and including day d, enforcing temporal integrity. The statistics are: in-degree (number of incoming relationships), out-degree (number of outgoing relationships), unique in-neighbour count, unique out-neighbour count, relationship-type diversity (total degree divided by the number of distinct neighbours, measuring how concentrated the connectivity is), mention count (number of articles in the window that reference the entity), days since first mention (an entity recency proxy), and total degree. These per-entity vectors are mean-aggregated across all entities mentioned in the window to produce a topology vector $\mathbf{t}_d \in \mathbb{R}^8$. Unlike the path and subgraph blocks, topology features are schema-agnostic by construction: the underlying query counts relationships without filtering by candidate tag or inspecting relationship types, so $\mathbf{t}_d$ is identical across ontology candidates and any AUC difference between candidates cannot be attributed to it.

Feature modes. The path, subgraph, and topology blocks can be combined according to one of three feature modes: `path` ($\mathbf{x}_d = [\,\mathbf{p}_d \,\|\, \mathbf{t}_d\,] \in \mathbb{R}^{392}$), `subgraph` ($\mathbf{x}_d = [\,\mathbf{s}_d \,\|\, \mathbf{t}_d\,] \in \mathbb{R}^{104}$), and `hybrid` ($\mathbf{x}_d = [\,\mathbf{p}_d \,\|\, \mathbf{s}_d \,\|\, \mathbf{t}_d\,] \in \mathbb{R}^{488}$), which concatenates all three. The sweep (Section 5.4) fixes `hybrid` throughout rather than varying feature mode.

4.6 Downstream Classifier and Evaluation Protocol

Temporal split Days are sorted in chronological order and partitioned 70/30: the first 70 % form the training set and the remaining 30 % the validation set. The partition is

applied by day and never shuffled, ensuring that no future information enters the training set.

Classifier An XGBoost classifier (Chen and Guestrin, 2016) is trained on the stacked day-level feature vectors with the following hyperparameters: 100 trees, `max_depth` = 3, learning rate 0.1, row and column subsampling rates 0.9. Validation AUC-ROC is the primary selection metric returned to the Evolution Agent; F1 at threshold 0.5 is reported alongside it.

Text-only baseline Before any LLM-constructed KG candidate is evaluated, a text-only baseline is established: mean-pooled article text embeddings over the lookback window, with no graph structure. The same day set and temporal split are used, isolating the contribution of KG construction and ontology design from the predictive content of raw article text. The baseline AUC is the reference against which all KG-based candidates are compared, and against which the hypotheses in Section 3 are tested.

5 Experimental Setup

5.1 Data

We use a subset of the FNSPID dataset (Dong et al., 2024) covering two Nasdaq tickers, MSFT and TSLA, with full article text, headline, and timestamp for each article, at a fixed rate of 4 articles/day per ticker. For each ticker we define a common in-sample window of 200 calendar days, from 2 May 2022 to 18 November 2022, used for the ontology-evolution sweep and the temporal train/validation split defined in Section 4.6, and hold out a subsequent 100-day out-of-sample window, from 19 November 2022 to 26 February 2023, that is not seen during ontology search and is reserved for final evaluation of the selected ontology. The window starts at 2 May 2022, where both tickers reach consistently dense article volume; earlier months are excluded due to sparse coverage. Figure 3 shows adjusted close price and daily article volume for each ticker over the full history, with the in-sample and out-of-sample windows overlaid.

5.2 Baselines

We compare against two reference conditions, both targeted by H1. The text-only baseline, described in Section 4.6, isolates the contribution of graph structure by replacing KG-derived features with mean-pooled article text embeddings under an identical day set and temporal split. The static-ontology baseline is an independently constructed graph, built by the same KG Builder Agent and LLM extraction pipeline as the task-aware candidates (Section 4.3) using the default construction method, but never revised by the

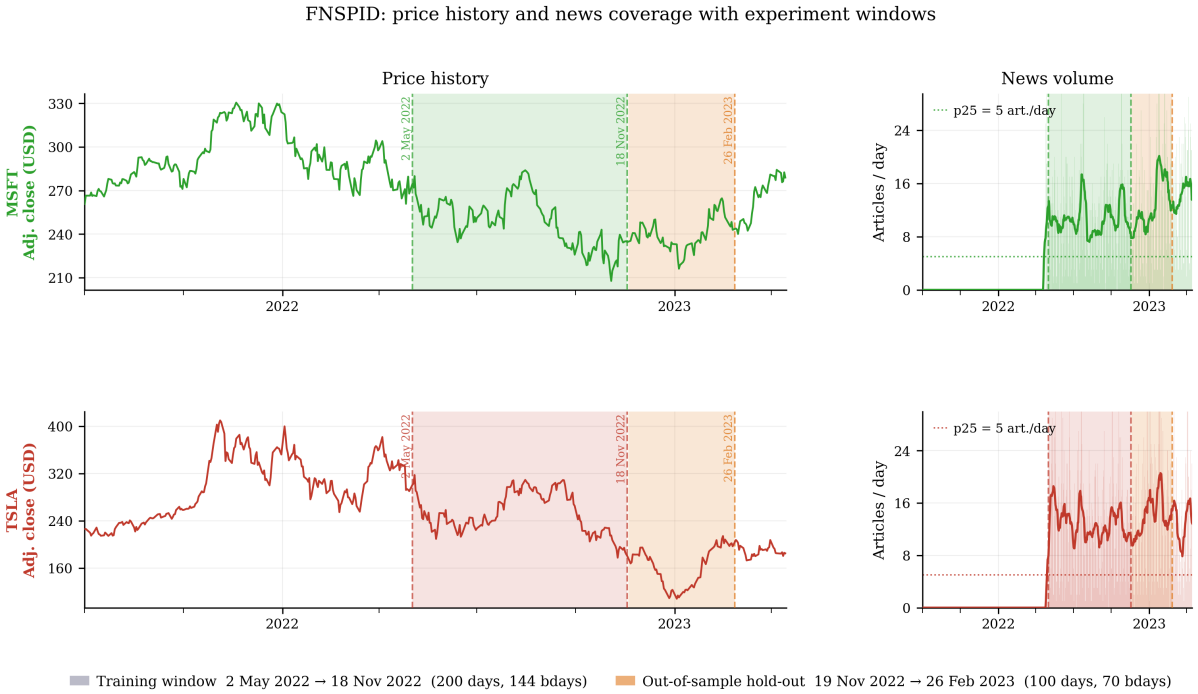

Figure 3: FNSPID price history and news coverage, with experiment windows.

Ontology Evolution Agent in response to the downstream feedback signal; this isolates the contribution of the feedback loop itself from that of KG construction in general.

5.3 Models

The pipeline separates two LLM roles that differ in required capacity and call volume.

Extraction SLM The KG Builder Agent (Section 4.3) issues one extraction call per article and is the highest-volume LLM consumer in the loop. We serve it with an AWQ-quantized small language model, `Qwen2.5-7B-Instruct-AWQ`, through a self-hosted vLLM server exposing an OpenAI-compatible API on a single GPU node (8192-token context), trading model capacity for the throughput needed to run many candidates in parallel under the sweep budget.

Ontology Evolution LLM The Ontology Evolution Agent (Section 4.4) issues one proposal call per step and reasons over the aggregated feedback signal from all candidates at that step, a lower-volume, higher-reasoning task. We serve this role with `qwen.qwen3-32b-v1:0` via AWS Bedrock.

Embeddings Article and node text embeddings (feature construction and the text-only baseline) use `sentence-transformers/all-MiniLM-L6-v2`, served through Hugging Face Text Embeddings Inference (TEI) on a CPU node.

Dimension	Levels
Evolution steps	3, 5, 7
Lookback window (days)	3, 8, 10, 20
Chain hops (min = max)	3, 5, 6
Evolution prompt	`default`, `fundamental`, `event-driven`, `macro-context`

Table 1: Dimensions crossed in the balanced factorial sweep, further crossed with ticker (MSFT, TSLA).

5.4 Sweep Configuration

Rather than a full grid search, the sweep uses a balanced factorial design: a mixed orthogonal array (Lin and Stufken, 2025), built so that every pair of factor levels among the four dimensions in Table 1 occurs together equally often. This makes each dimension's effect readable on its own, without being distorted by the others, while needing far fewer runs than testing every combination would — designs of this kind have been shown to be effective specifically for exploring machine-learning hyperparameter spaces (Shi et al., 2023). The design is crossed with two tickers, MSFT and TSLA. Candidates per step is fixed at 3, articles per day at 4, and feature mode at `hybrid` (Section 4.5) throughout.

The four evolution-prompt variants bias the Ontology Evolution Agent toward different semantic categories when proposing new node and relationship types. Each ticker receives its own independently-balanced 36-configuration design over the remaining four dimensions, for 72 configurations in total (36 MSFT, 36 TSLA). Because raw predictive AUC differs systematically between the two tickers (they pose different underlying prediction-difficulty tasks), two complementary analyses are supported. Per-ticker, each 36-configuration design is independently balanced on its own: steps and chain-hops effects, the two dimensions of primary interest, are read directly from per-group means, while lookback and evolution-prompt effects require OLS regression jointly on all four within-ticker dimensions, since the Latin-square construction leaves a small number of lookback–prompt cells unobserved within each ticker. Pooling both tickers supports the same distinction with ticker added as a covariate: per-group means for steps and chain-hops, OLS regression for lookback and evolution-prompt, with ticker-level differences absorbed as a covariate and cancelling out of marginal contrasts along the other dimensions. Figure 4 verifies this directly against the realized 72-configuration design: every pair of dimensions is perfectly balanced (a uniform count in every cell) except lookback–prompt, where the Latin-square construction leaves exactly one cell empty per lookback level, rotating across prompts — the source of the one exception noted above.

5.5 Infrastructure

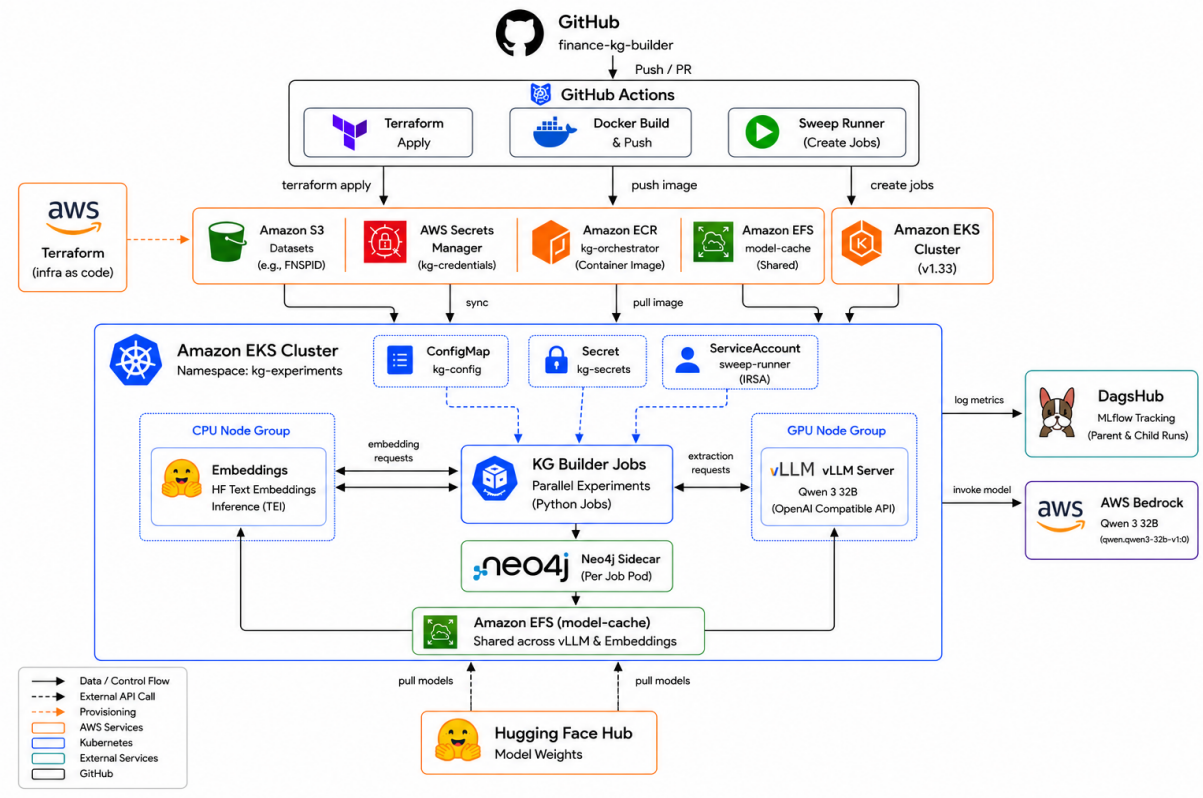

Figure 5: Experimental infrastructure.

The ontology-evolution sweep is run on the cloud pipeline shown in Figure 5: a push to the `finance-kg-builder` repository triggers a GitHub Actions workflow that provisions the Amazon EKS cluster via Terraform, builds and pushes the orchestrator image. A separate dispatch workflow launches the sweep as parallel Kubernetes jobs, each pairing a KG Builder job pod with a Neo4j sidecar, a CPU node group serving embeddings, and a GPU node group serving the extraction SLM (Section 5.3); run metrics are logged to DagsHub MLflow, and ontology-proposal calls are routed to AWS Bedrock.

6 Results

[TODO: Write this section]

7 Discussion

[TODO: Write this section]

8 Conclusion

[TODO: Write this section]

References

Abhinav Arun, Fabrizio Dimino, Tejas Prakash Agarwal, Bhaskarjit Sarmah, and Stefano Pasquali. FinReflectKG: Agentic construction and evaluation of financial knowledge graphs. In *Proceedings of the 6th ACM International Conference on AI in Finance*, 2025. URL `https://arxiv.org/abs/2508.17906`.

Peter Bloem, Xander Wilcke, Lucas van Berkel, and Victor de Boer. kgbench: A collection of knowledge graph datasets for evaluating relational and multimodal machine learning. In *The Semantic Web – 18th International Conference, ESWC 2021*, Lecture Notes in Computer Science. Springer, 2021. doi: 10.1007/978-3-030-77385-4_37.

Tomaz Bratanic. Building knowledge graphs with LLM graph transformer, 2024. URL `https://medium.com/data-science/building-knowledge-graphs-with-llm-graph-transformer-a91045c49b59`.

Harrison Chase. LangChain, 2022. URL `https://github.com/langchain-ai/langchain`.

Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. URL `https://arxiv.org/abs/1603.02754`.

Tamara Cucumides and Floris Geerts. From features to structure: Task-aware graph construction for relational and tabular learning with GNNs. In *3rd International Workshop on Tabular Data Analysis (TaDA) at VLDB 2025*, 2025. URL `https://www.vldb.org/2025/Workshops/VLDB-Workshops-2025/TaDA/TaDA25_5.pdf`.

Xiao Ding, Yue Zhang, Ting Liu, and Junwen Duan. Deep learning for event-driven stock prediction. *IJCAI*, 2015.

Xiao Ding, Yue Zhang, Ting Liu, and Junwen Duan. Knowledge-driven event embedding for stock prediction. In *Proceedings of the 26th International Conference on Computational Linguistics (COLING)*, pages 2133–2142, 2016. URL `https://aclanthology.org/C16-1201/`.

Yuxiao Dong, Nitesh V. Chawla, and Ananthram Swami. metapath2vec: Scalable representation learning for heterogeneous networks. In *KDD*, 2017.

Zihan Dong, Xinyu Fan, and Zhiyuan Peng. FNSPID: A comprehensive financial news dataset in time series. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024. doi: 10.1145/3637528.3671629. URL https://dl.acm.org/doi/10.1145/3637528.3671629.

Nicolas Heist, Sven Hertling, and Heiko Paulheim. KGrEaT: A framework to evaluate knowledge graphs via downstream tasks. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM)*, 2023. doi: 10.1145/3583780.3615241. URL https://arxiv.org/abs/2308.10537.

Ziniu Hu, Weiqing Liu, Jiang Bian, Xuanzhe Liu, and Tie-Yan Liu. Listening to chaotic whispers: A deep learning framework for news-oriented stock trend prediction. In *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining (WSDM)*, pages 261–269, 2018. URL https://arxiv.org/abs/1712.02136.

Shaoxiong Ji, Shirui Pan, Erik Cambria, Pekka Marttinen, and Philip S. Yu. A survey on knowledge graph embedding: Approaches and applications. *IEEE Transactions on Knowledge and Data Engineering*, 33(4):1–19, 2021.

Jiayi Li, Daniel Garijo, and María Poveda-Villalón. Large language models for ontology engineering: A systematic literature review. 2025. in press; prepared using sagej.cls, DOI to be assigned by SAGE.

C. Devon Lin and John Stufken. Orthogonal arrays: A review. *WIREs Computational Statistics*, 2025. doi: 10.1002/wics.70029.

Jerry Liu. LlamaIndex, 2022. URL https://github.com/run-llama/llama_index.

LlamaIndex. Comparing LLM path extractors for knowledge graph construction, 2024. URL https://developers.llamaindex.ai/python/examples/property_graph/dynamic_kg_extraction/.

Belinda Mo, Kyssen Yu, Joshua Kazdan, Joan Cabezas, Proud Mpala, Lisa Yu, Chris Cundy, Charilaos Kanatsoulis, and Sanmi Koyejo. KGGen: Extracting knowledge graphs from plain text with language models, 2025. URL https://openreview.net/forum?id=YyhRJXxbpi.

Jindřich Mynarz and Kateřina Haniková. Test-driven knowledge graph construction. In *Proceedings of the Knowledge Graph Construction Workshop (KGC 2023)*, volume 3471 of *CEUR Workshop Proceedings*, 2023. URL https://ceur-ws.org/Vol-3471/paper4.pdf.

Neo4j. User guide: Knowledge graph builder — *neo4j-graphrag-python*, 2024. URL https://neo4j.com/docs/neo4j-graphrag-python/current/user_guide_kg_builder.html.

Neo4j Labs. Neo4j LLM knowledge graph builder, 2024. URL https://neo4j.com/labs/genai-ecosystem/llm-graph-builder/.

Shirui Pan et al. Large language models meet knowledge graphs: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 2024.

Heiko Paulheim. Knowledge graph refinement: A survey of approaches and evaluation methods. *Semantic Web*, 8(3):489–508, 2017.

Heiko Paulheim. Structural quality metrics for knowledge graphs. In *Knowledge Graphs: Methodology, Tools and Selected Use Cases*. Springer, 2024. TODO: verify exact chapter/pages.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992, 2019. URL https://arxiv.org/abs/1908.10084.

Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *ESWC*, 2018.

Chenlu Shi, Ashley Kathleen Chiu, and Hongquan Xu. Evaluating designs for hyperparameter tuning in deep neural networks. *The New England Journal of Statistics in Data Science*, 1(3):334–341, 2023. doi: 10.51387/23-NEJSDS26.

Chuan Shi, Yitong Li, Jiawei Zhang, Yizhou Sun, and Philip S. Yu. A survey of heterogeneous information network analysis. In *IEEE TKDE*, 2017.

Yizhou Sun, Jiawei Han, Xifeng Yan, Philip S. Yu, and Tianyi Wu. Pathsim: Meta path-based top-k similarity search in heterogeneous information networks. In *VLDB*, 2011.

Rakshit Trivedi, Hanjun Dai, Yichen Wang, and Le Song. Know-evolve: Deep temporal reasoning for dynamic knowledge graphs. *ICML*, 2017.

Hong Ting Tsang, Jiaxin Bai, Haoyu Huang, Qiao Xiao, Tianshi Zheng, Baixuan Xu, Shujie Liu, and Yangqiu Song. AutoGraph-R1: End-to-end reinforcement learning for knowledge graph construction, 2025. URL https://arxiv.org/abs/2510.15339.

Quan Wang, Zhendong Mao, Bin Wang, and Li Guo. Knowledge graph embedding: A survey of approaches and applications. *IEEE Transactions on Knowledge and Data Engineering*, 29(12):2724–2743, 2017.

Yumo Xu and Shay B. Cohen. Stock movement prediction from tweets and historical prices. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, pages 1970–1979, 2018. URL `https://aclanthology.org/P18-1183/`.

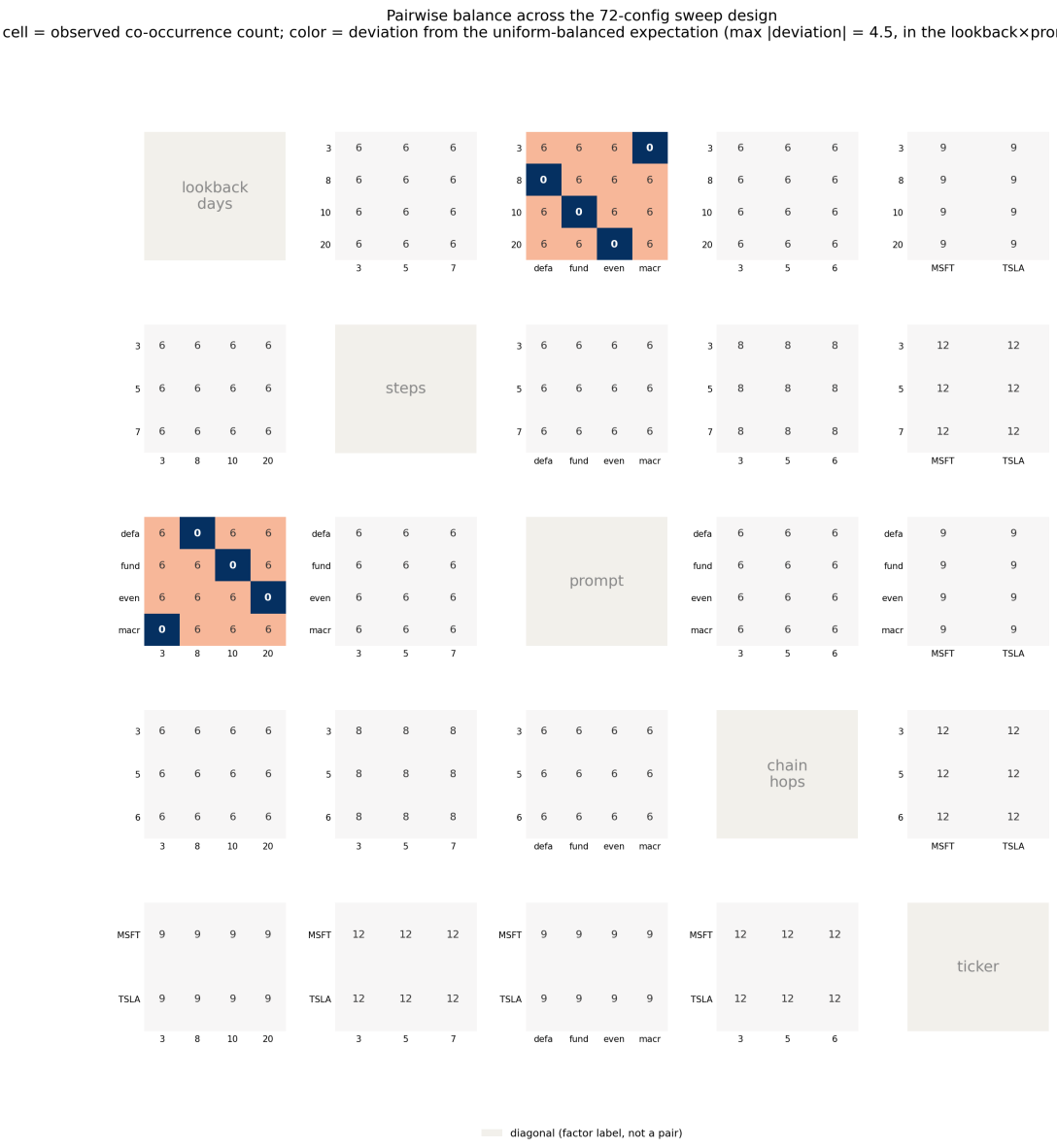

Figure 4: Pairwise co-occurrence counts across all ten dimension pairs in the realized 72-configuration design. Nine pairs are perfectly uniform; lookback–prompt (top row, third panel) has one empty cell per lookback level, rotating across the four prompts.